Meilisearch for Olam Agri: Verified Integration Report

Prepared for presentation | Date: 30 Mar 2026

Scope: This report validates and condenses the two provided reference documents: “Meilisearch Integration and Requirements.pdf” and “Meilisearch Integration for Olam Agri.pdf”. Only information cross-checked against official Meilisearch documentation is treated as confirmed.

Executive Summary

- Meilisearch can fit Olam Agri use cases where fast, user-facing search is required (products, farmer profiles, sustainability records, catalog and dashboard search).
- Performance impact is real during indexing: indexing is CPU- and memory-intensive and can reduce query performance if not tuned.
- A safe enterprise rollout should use a middleware layer between SAP/operational systems and Meilisearch for denormalization, security, and controlled batch ingestion.
- Community Edition is suitable for a single-instance setup; sharding + replication for horizontal scale and high availability require Enterprise Edition.
- Main blockers are architectural and operational (single-node limitations in CE, data modeling from SAP, security governance, and operations discipline).

1) Is performance likely to go down because of Meilisearch?

Short answer: It can, during indexing or poorly tuned updates. Search latency can stay low if indexing resources are bounded and ingestion is designed properly.

- Official docs confirm indexing is memory-intensive and multi-threaded.
- By default, Meilisearch may use up to ~2/3 of available RAM for indexing.
- By default, indexing uses up to ~1/2 of available processing threads.
- If memory/thread limits are too high, search quality and responsiveness can degrade during indexing.
- If limits are too low, ingestion takes longer and data freshness suffers.

Practical guardrails for Olam Agri: set --max-indexing-memory and --max-indexing-threads, avoid huge single payloads, and ingest updates in controlled batches.

2) How to integrate Meilisearch to Olam Agri (recommended architecture)

1. **Step 1: Define search domains** (what needs fast search): e.g., farmer master data, lot/commodity references, sustainability/traceability records, customer portal content.
2. **Step 2: Build search-ready documents**: flatten relational SAP entities into denormalized JSON per domain.
3. **Step 3: Add sync pipeline**: use CDC/event streams or scheduled incremental jobs to feed Meilisearch tasks.
4. **Step 4: Harden security**: backend owns admin/search keys; frontend receives short-lived tenant tokens only.
5. **Step 5: Tune indexing**: batch sizing + indexing memory/thread limits + low-traffic reindex windows.
6. **Step 6: Operate safely**: enable dumps/snapshots, monitor task queues, define rollback and reindex playbooks.

3) Requirements (validated)

Area	Minimum expectation for production readiness
Runtime & OS	Supported Linux/macOS/Windows Server options; Docker or binary deployment
Security baseline	Production mode with master key (>=16 bytes); key lifecycle managed by backend
Sizing baseline	Disk capacity should be planned with generous headroom (official FAQ suggests starting arou
Indexing controls	Configure max indexing memory and indexing threads to prevent resource starvation
Data operations	Use batch ingestion and task monitoring; keep backup/restore strategy (dumps/snapshots)
Network/HA	Single-node for CE; Enterprise needed for sharding and replication

4) Self-hosting options

- **Option A — Community Edition (MIT):** Best for single-instance deployment and moderate workloads.
- **Option B — Enterprise Edition (BUSL):** Required for sharding and replication (horizontal scale + HA).
- **Option C — Meilisearch Cloud:** Managed operations, fast rollout, less infra burden.

For strict data control policies, self-hosting (A or B) is usually preferred. For faster time-to-value and lower ops overhead, Cloud is usually preferred.

5) Key blockers and risks

- **Horizontal scaling/HA in CE:** not available; CE is single-instance focused.
- **Data modeling effort:** SAP data often requires substantial denormalization and schema shaping for search relevance.
- **Near-real-time sync complexity:** CDC/event reliability and idempotent upserts must be engineered.
- **Security and tenancy governance:** tenant token generation and key rotation must be implemented on backend.
- **Operational maturity:** monitoring, backup, recovery drills, and version upgrade process are required.

6) 90-day implementation plan (practical)

1. Weeks 1-2: scope use cases, define KPIs (search latency, indexing SLA, freshness target), and select edition (CE/EE/Cloud).
2. Weeks 3-4: build PoC for one domain + baseline sizing + security model.
3. Weeks 5-8: add middleware pipelines, CDC/incremental ingestion, relevance tuning, and tenant token flow.
4. Weeks 9-12: load/performance testing, backup/restore drills, production readiness checklist, staged rollout.

What was corrected from the reference PDFs

The provided PDFs contain many useful ideas, but also some over-assertive statements. In this report, claims were filtered to keep only those supported by official Meilisearch docs (for limits, deployment behavior, security model, and edition differences). Organization-specific assumptions (exact Olam Agri platform internals, guaranteed latency numbers, and external vendor details) were treated as assumptions unless independently verifiable.

Verified source list (primary)

- Meilisearch Documentation – FAQ
- Meilisearch Documentation – Known limitations
- Meilisearch Documentation – Configuration reference
- Meilisearch Documentation – Impact of RAM and multi-threading on indexing performance
- Meilisearch Documentation – Install Meilisearch locally
- Meilisearch Documentation – Using Meilisearch with Docker
- Meilisearch Documentation – Security and tenant tokens
- Meilisearch Documentation – Enterprise and Community editions
- Meilisearch Documentation – Replication and sharding overview

Reference inputs reviewed: “Meilisearch Integration and Requirements.pdf” and “Meilisearch Integration for Olam Agri.pdf”.